

ADA Simplified Lab Programs

Merge Sort:

— □ ×

```
#include <stdio.h>

void merge(int a[], int low, int mid, int high) {
    int i = low, j = mid + 1, k = 0, b[100];
    while (i <= mid && j <= high)
        b[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i <= mid)
        b[k++] = a[i++];
    while (j <= high)
        b[k++] = a[j++];
    for (i = low, j = 0; i <= high; i++, j++)
        a[i] = b[j];
}

void mergesort(int a[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        mergesort(a, low, mid);
        mergesort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
}

int main() {
    int a[100], n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    mergesort(a, 0, n - 1);
    printf("Sorted array:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    return 0;
}
```

Quick Sort:

— □ ×

```
#include <stdio.h>

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int partition(int a[], int low, int high) {
    int pivot = a[low];
    int i = low, j = high;
    while (i < j) {
        while (a[i] <= pivot && i <= high) i++;
        while (a[j] > pivot && j >= low) j--;
        if (i < j)
            swap(&a[i], &a[j]);
    }
    swap(&a[low], &a[j]);
    return j;
}

void quicksort(int a[], int low, int high) {
    if (low < high) {
        int p = partition(a, low, high);
        quicksort(a, low, p - 1);
        quicksort(a, p + 1, high);
    }
}

int main() {
    int a[100], n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    quicksort(a, 0, n - 1);
    printf("Sorted array:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    return 0;
}
```

Selection Sort:

— □ ×

```
#include <stdio.h>

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

void selectionSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[min])
                min = j;
        }
        if (min != i)
            swap(&a[i], &a[min]);
    }
}

int main() {
    int a[100], n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    selectionSort(a, n);
    printf("Sorted array:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    return 0;
}
```

Topological Sort:

— □ ×

```
#include <stdio.h>

int a[10][10], indegree[10], n;

void findIndegree() {
    for (int j = 0; j < n; j++) {
        indegree[j] = 0;
        for (int i = 0; i < n; i++)
            indegree[j] += a[i][j];
    }
}

void topoSort() {
    int stack[10], top = -1, result[10], k = 0;

    findIndegree();
    for (int i = 0; i < n; i++)
        if (indegree[i] == 0)
            stack[++top] = i;

    while (top != -1) {
        int u = stack[top--];
        result[k++] = u;
        for (int v = 0; v < n; v++) {
            if (a[u][v]) {
                indegree[v]--;
                if (indegree[v] == 0)
                    stack[++top] = v;
            }
        }
    }

    printf("Topological Order: ");
    for (int i = 0; i < k; i++)
        printf("%d ", result[i]);
}

int main() {
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &a[i][j]);

    topoSort();
    return 0;
}
```

Dijkstra's Algorithm:

— □ ×

```
#include <stdio.h>

void dijkstra(int cost[10][10], int n, int src, int dist[]) {
    int visited[10] = {0}, min, u;

    for (int i = 0; i < n; i++)
        dist[i] = cost[src][i];

    visited[src] = 1;

    for (int i = 1; i < n; i++) {
        min = 999;
        for (int j = 0; j < n; j++)
            if (!visited[j] && dist[j] < min) {
                min = dist[j];
                u = j;
            }

        visited[u] = 1;

        for (int v = 0; v < n; v++)
            if (!visited[v] && dist[u] + cost[u][v] < dist[v])
                dist[v] = dist[u] + cost[u][v];
    }
}

int main() {
    int n, cost[10][10], dist[10], src;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost adjacency matrix (use 999 for no edge):\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &cost[i][j]);

    printf("Enter source vertex (0 to %d): ", n - 1);
    scanf("%d", &src);

    dijkstra(cost, n, src, dist);

    printf("Shortest distances from source %d:\n", src);
    for (int i = 0; i < n; i++)
        printf("To %d = %d\n", i, dist[i]);

    return 0;
}
```

Kruskal's Algorithm:

— □ ×

```
#include <stdio.h>

int parent[10];

int find(int i) {
    while (parent[i])
        i = parent[i];
    return i;
}

void union_set(int i, int j) {
    parent[j] = i;
}

int main() {
    int cost[10][10], n, i, j, u, v, a, b, min, edges = 1, mincost = 0;
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter cost adjacency matrix (0 if no edge):\n");
    for (i = 1; i <= n; i++)
        for (j = 1; j <= n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0)
                cost[i][j] = 999;
        }

    printf("Edges in Minimum Cost Spanning Tree:\n");
    while (edges < n) {
        min = 999;
        for (i = 1; i <= n; i++)
            for (j = 1; j <= n; j++)
                if (cost[i][j] < min) {
                    min = cost[i][j];
                    a = u = i;
                    b = v = j;
                }

        u = find(u);
        v = find(v);

        if (u != v) {
            union_set(u, v);
            printf("%d Edge (%d,%d) = %d\n", edges++, a, b, min);
            mincost += min;
        }
        cost[a][b] = cost[b][a] = 999;
    }

    printf("Total cost of MST = %d\n", mincost);
    return 0;
}
```

Prim's Algorithm:

— □ ×

```
#include <stdio.h>

int main() {
    int cost[10][10], visited[10] = {0}, n, i, j, min, u, v, edges = 1, mincost = 0;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter cost adjacency matrix (0 if no edge):\n");
    for (i = 1; i <= n; i++)
        for (j = 1; j <= n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0)
                cost[i][j] = 999;
        }

    visited[1] = 1; // Start from vertex 1
    printf("Edges in Minimum Cost Spanning Tree:\n");

    while (edges < n) {
        min = 999;
        for (i = 1; i <= n; i++) {
            if (visited[i]) {
                for (j = 1; j <= n; j++) {
                    if (!visited[j] && cost[i][j] < min) {
                        min = cost[i][j];
                        u = i;
                        v = j;
                    }
                }
            }
        }
        printf("%d) Edge (%d,%d) = %d\n", edges++, u, v, min);
        mincost += min;
        visited[v] = 1;
        cost[u][v] = cost[v][u] = 999;
    }

    printf("Total cost of MST = %d\n", mincost);
    return 0;
}
```

0/1 Knapsack DP:

— □ ×

```
#include <stdio.h>

int w[10], p[10], dp[10][10], x[10];
int n, capacity;

int max(int a, int b) {
    return (a > b) ? a : b;
}

void knapsack() {
    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= capacity; j++) {
            if (i == 0 || j == 0)
                dp[i][j] = 0;
            else if (w[i] > j)
                dp[i][j] = dp[i - 1][j];
            else
                dp[i][j] = max(dp[i - 1][j], dp[i - 1][j - w[i]] + p[i]);
        }
    }
}

void findItems() {
    int i = n, j = capacity;
    while (i > 0 && j > 0) {
        if (dp[i][j] != dp[i - 1][j]) {
            x[i] = 1;
            j -= w[i];
        }
        i--;
    }
}

int main() {
    printf("Enter number of items: ");
    scanf("%d", &n);
    printf("Enter knapsack capacity: ");
    scanf("%d", &capacity);

    printf("Enter weights:\n");
    for (int i = 1; i <= n; i++)
        scanf("%d", &w[i]);

    printf("Enter profits:\n");
    for (int i = 1; i <= n; i++)
        scanf("%d", &p[i]);

    knapsack();
    findItems();

    printf("Items included:\n");
    printf("Item\tWeight\tProfit\n");
    for (int i = 1; i <= n; i++)
        if (x[i])
            printf("%d\t%d\t%d\n", i, w[i], p[i]);

    printf("Max Profit = %d\n", dp[n][capacity]);
    return 0;
}
```


Discrete & Continuous Knapsack (Greedy Method):

— □ ×

```
#include <stdio.h>

void greedyKnapsack(int n, float w[], float p[], float capacity) {
    float x[20] = {0}, ratio[20], totalProfit = 0;
    int i, j;

    // Compute profit/weight ratio
    for (i = 0; i < n; i++)
        ratio[i] = p[i] / w[i];

    // Sort items by ratio (descending)
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (ratio[i] < ratio[j]) {
                float temp;

                // Swap ratio
                temp = ratio[i]; ratio[i] = ratio[j]; ratio[j] = temp;

                // Swap weights
                temp = w[i]; w[i] = w[j]; w[j] = temp;

                // Swap profits
                temp = p[i]; p[i] = p[j]; p[j] = temp;
            }
        }
    }

    // Fill the knapsack
    for (i = 0; i < n; i++) {
        if (w[i] <= capacity) {
            x[i] = 1.0; // Take full item
            totalProfit += p[i];
            capacity -= w[i];
        } else {
            x[i] = capacity / w[i]; // Take fraction
            totalProfit += x[i] * p[i];
            break; // Knapsack is full
        }
    }

    // Result
    printf("\nItem\tWeight\tProfit\tFraction taken\n");
    for (i = 0; i < n; i++)
        printf("%d\t%.1f\t%.1f\t%.2f\n", i + 1, w[i], p[i], x[i]);

    printf("\nTotal Profit = %.2f\n", totalProfit);
}

int main() {
    int n;
    float w[20], p[20], capacity;

    printf("Enter number of items: ");
    scanf("%d", &n);

    printf("Enter knapsack capacity: ");
    scanf("%f", &capacity);

    printf("Enter weights and profits:\n");
    for (int i = 0; i < n; i++)
        scanf("%f%f", &w[i], &p[i]);

    greedyKnapsack(n, w, p, capacity);
    return 0;
}
```

Floyd's Algorithm:

— □ ×

```
#include <stdio.h>

int min(int a, int b) {
    return (a < b) ? a : b;
}

void floyd(int dist[10][10], int n) {
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
}

int main() {
    int n, dist[10][10];
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix (999 for no direct edge):\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &dist[i][j]);

    floyd(dist, n);

    printf("All-Pairs Shortest Paths:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            printf("%d ", dist[i][j]);
        printf("\n");
    }

    return 0;
}
```

Warshall's Algorithm:

— □ ×

```
#include <stdio.h>

void warshall(int p[10][10], int n) {
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                p[i][j] = p[i][j] || (p[i][k] && p[k][j]);
}

int main() {
    int n, a[10][10];
    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix (1 if edge exists, else 0):\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &a[i][j]);

    warshall(a, n);

    printf("Transitive Closure (Path Matrix):\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            printf("%d ", a[i][j]);
        printf("\n");
    }

    return 0;
}
```

N-Queens:

— □ ×

```
#include <stdio.h>

int board[10], count = 0;

int isSafe(int row, int col) {
    for (int i = 0; i < row; i++)
        if (board[i] == col || abs(board[i] - col) == abs(i - row))
            return 0;
    return 1;
}

void display(int n) {
    printf("\nSolution %d:\n", ++count);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            printf("%c ", board[i] == j ? 'Q' : '*');
        printf("\n");
    }
}

void solve(int row, int n) {
    for (int col = 0; col < n; col++) {
        if (isSafe(row, col)) {
            board[row] = col;
            if (row == n - 1)
                display(n);
            else
                solve(row + 1, n);
        }
    }
}

int main() {
    int n;
    printf("Enter number of queens: ");
    scanf("%d", &n);
    solve(0, n);
    printf("Total solutions = %d\n", count);
    return 0;
}
```

Subset Sum Problem:

— □ ×

```
#include <stdio.h>

int w[10], x[10], n, target;

void subset(int s, int k, int r) {
    x[k] = 1;
    if (s + w[k] == target) {
        printf("Subset: ");
        for (int i = 0; i <= k; i++)
            if (x[i])
                printf("%d ", w[i]);
        printf("\n");
    } else if (s + w[k] + w[k + 1] <= target)
        subset(s + w[k], k + 1, r - w[k]);

    if ((s + r - w[k] >= target) && (s + w[k + 1] <= target)) {
        x[k] = 0;
        subset(s, k + 1, r - w[k]);
    }
}

int main() {
    int sum = 0;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements in ascending order:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &w[i]);
        sum += w[i];
    }

    printf("Enter target sum: ");
    scanf("%d", &target);

    if (sum < target || w[0] > target) {
        printf("No solution possible.\n");
        return 0;
    }

    subset(0, 0, sum);
    return 0;
}
```